

TP3 – APIs REST : client robuste, Pydantic et géocodage

M1 Data & IA – Université Catholique de Lille

Rendu attendu

- Un notebook Jupyter `.ipynb` exécutable de bout en bout.
- Travail en binôme conseillé.
- Pour chaque question, la réponse doit apparaître dans une cellule **Markdown** placée sous le code.
- Nom du fichier : `TP3_API_NOM_Prenom.ipynb`

Contexte

La foncière lilloise veut franchir un cap : passer d’une analyse statique de fichiers téléchargés à un **pipeline de données enrichi par des APIs** capable de tourner en production chaque nuit.

Votre responsable a trois demandes concrètes. Premièrement, automatiser la récupération des données de population des communes via l’API officielle `geo.api.gouv.fr`, en remplaçant la jointure manuelle sur fichier CSV de votre TP2. Deuxièmement, **géocoder les adresses DVF** – c’est-à-dire obtenir les coordonnées GPS de chaque transaction – afin de produire des cartes interactives. Troisièmement, structurer tout cela dans un **pipeline robuste** : avec gestion des erreurs, cache, retry et validation des données reçues.

Ce TP vous fait donc passer de « *ça fonctionne sur mon poste* » à « *ça tourne en production à 2h du matin sans que je sois réveillé* ».

Jeux de données et APIs

- **Source 1** : `dvf_clean.parquet` ou `dvf_enrichi.parquet` – votre fichier Parquet produit en TP1/TP2.
Si absent : utilisez le bloc de secours fourni en section 1.
- **API 1** : API Découpage Administratif – `geo.api.gouv.fr`
Pas de clé requise. Communes du Nord : `/departements/59/communes?fields=nom,code,population&format=`
- **API 2** : Nominatim OpenStreetMap – `nominatim.openstreetmap.org`
Pas de clé requise. User-Agent obligatoire. Limite stricte : **1 requête par seconde**.

Objectifs pédagogiques

À l’issue du TP, vous devez être capables de :

- construire un client HTTP robuste avec `Session`, `Retry`, `timeout` et `raise_for_status()` ;
- mesurer expérimentalement le gain d’une `Session` persistante sur des appels multiples ;
- modéliser et valider les réponses d’une API avec Pydantic (champs optionnels, validateurs, `fail soft`) ;
- mettre en cache des résultats d’API avec `requests-cache` et un cache fichier JSON ;
- géocoder des adresses textuelles en coordonnées GPS via Nominatim en respectant les contraintes de rate limit ;
- assembler un pipeline de données enrichi et produire une carte interactive avec `folium`.

1. Mise en place

1.1. Installations et imports

```
pip install requests requests-cache pydantic python-dotenv folium tqdm unicode
```

```
import requests
import requests_cache
import time, os, json, warnings
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import folium
from pathlib import Path
from tqdm import tqdm
from pydantic import BaseModel, Field, field_validator, ValidationError
from typing import Optional
from datetime import date
from dotenv import load_dotenv
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

load_dotenv()
warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 50)
pd.set_option("display.float_format", "{:,.0f}".format)

print("Imports OK")
```

1.2. Chargement du dataset

Deux chemins selon votre situation

- **Vous avez un Parquet issu du TP1 ou TP2** : utilisez le Chemin A.
- **Fichier absent** : utilisez le Chemin B (pipeline autonome complet).

Chemin A

```
# Priorit : fichier enrichi TP2, sinon fichier propre TP1
if Path("dvf_enrichi.parquet").exists():
    df = pd.read_parquet("dvf_enrichi.parquet")
elif Path("dvf_clean.parquet").exists():
    df = pd.read_parquet("dvf_clean.parquet")
else:
    raise FileNotFoundError("Aucun Parquet trouv -- utiliser le Chemin B")

print(f"Dataset charge : {df.shape[0]:,} lignes x {df.shape[1]} colonnes")
df.head(3)
```

Chemin B – Pipeline autonome

```
# =====
# CHEMIN B -- a executer si aucun Parquet n'est disponible
```

```
# =====
def extract(url):
    print("Telechargement DVF Nord 2023...")
    df = pd.read_csv(url, sep=";", low_memory=False)
    print(f" -> {df.shape[0]:,} lignes, {df.shape[1]} colonnes")
    return df

def transform(df):
    print(f"Transformation : {len(df):,} lignes en entree")
    df["date_mutation"] = pd.to_datetime(df["date_mutation"], errors="coerce")
    df["code_commune"] = df["code_commune"].astype(str)
    n = len(df)
    df = df.drop_duplicates().dropna(subset=["valeur_fonciere"])
    print(f" Doublons/NaN supprimes : {n - len(df)}")
    df["prix_m2"] = df["valeur_fonciere"] / df["surface_reelle_bati"]
    df = df[
        (df["valeur_fonciere"] > 0) &
        (df["valeur_fonciere"] < 5_000_000) &
        (df["prix_m2"].isna() | df["prix_m2"].between(100, 30_000))
    ].copy()
    print(f" -> {len(df):,} lignes en sortie")
    return df

def load(df, dest="dvf_clean.parquet"):
    df.to_parquet(dest, compression="snappy", index=False)
    print(f"Sauvegarde : {dest} ({os.path.getsize(dest)/1024:,.0f} Ko)")

URL = "https://files.data.gouv.fr/geo-dvf/latest/csv/2023/departements/59.csv.gz"
df_raw = extract(URL)
df = transform(df_raw)
load(df)
df = pd.read_parquet("dvf_clean.parquet")
print(f"\nDataset pret : {df.shape[0]:,} lignes x {df.shape[1]} colonnes")
# =====
```

Question 1. Vérifiez les types du DataFrame (`df.dtypes`). Corrigez `date_mutation` en `datetime64` si nécessaire. Affichez en Markdown : nombre total de transactions, période couverte (min et max de `date_mutation`), et les 5 types de biens les plus fréquents.

2. Client API robuste

2.1. Construire la Session avec Retry

Le cours a montré qu'un simple `requests.get()` est fragile : pas de timeout, pas de retry sur les erreurs temporaires, pas de headers communs. Vous allez construire un client robuste réutilisable.

```
GEO_BASE = "https://geo.api.gouv.fr"
HEADERS = {
    "User-Agent": "UCLille-TP3/1.0 (votre@email.ucl.fr)",
    "Accept": "application/json",
}

def creer_session(
    retries : int = 3,
    backoff : float = 1.0,
```

```

    status_codes : list = None,
) -> requests.Session:
    """
    Cree une Session requests avec retry automatique.
    Les erreurs 429, 500, 502, 503, 504 sont retentees.
    Pas de retry sur les erreurs clients (400, 401, 403, 404).
    """
    if status_codes is None:
        status_codes = [429, 500, 502, 503, 504]

    session = requests.Session()
    session.headers.update(HEADERS)

    retry = Retry(
        total = retries,
        backoff_factor = backoff,
        status_forcelist = status_codes,
        allowed_methods = ["GET", "HEAD"],
    )
    adapter = HTTPAdapter(max_retries=retry)
    session.mount("https://", adapter)
    session.mount("http://", adapter)

    return session

# Instanciatiion du client global
session = creer_session()
print("Session cree avec retry automatique")

```

Question 2. Expliquez en Markdown ce que fait le paramètre `backoff_factor=1.0`. Quel sera le temps d'attente entre la 1^{re} et la 2^e tentative ? Entre la 2^e et la 3^e ? Donnez la formule générale.

2.2. Premier appel – Communes du Nord

```

def get_communes_nord(session: requests.Session) -> list[dict]:
    """Recupere toutes les communes du departement 59 via geo.api.gouv.fr."""
    url = f"{GEO_BASE}/departements/59/communes"
    r = session.get(
        url,
        params = {"fields": "nom,code,population,codesPostaux", "format": "json"},
        timeout = 10,
    )
    r.raise_for_status()
    return r.json()

communes_raw = get_communes_nord(session)
print(f"Communes recuperees : {len(communes_raw)}")
print(f"Exemple : {communes_raw[0]}")

```

Question 3. Appelez `get_communes_nord()` et affichez le résultat. Combien de communes le département 59 contient-il ? Quels champs sont présents dans chaque item JSON ? La population est-elle toujours renseignée ?

2.3. Benchmark – Session vs appels unitaires

Le cours affirme qu'une Session persistante est significativement plus rapide que des appels indépendants sur de multiples requêtes. Vérifiez-le expérimentalement sur les communes du Nord.

```
import time

# Sous-liste de 50 communes pour le benchmark
codes_test = [c["code"] for c in communes_raw[:50]]
BASE_COMMUNE = f"{GEO_BASE}/communes"

def bench(func, n=3):
    """Mediane de n mesures, en millisecondes."""
    times = []
    for _ in range(n):
        t0 = time.perf_counter()
        func()
        times.append((time.perf_counter() - t0) * 1000)
    return round(sorted(times)[n // 2], 1)

def appels_unitaires():
    for code in codes_test:
        r = requests.get(f"{BASE_COMMUNE}/{code}", timeout=10)
        r.raise_for_status()

def appels_session():
    for code in codes_test:
        r = session.get(f"{BASE_COMMUNE}/{code}", timeout=10)
        r.raise_for_status()

t_unitaire = bench(appels_unitaires)
t_session = bench(appels_session)

print(f"Appels unitaires : {t_unitaire:>8.1f} ms")
print(f"Avec Session : {t_session:>8.1f} ms")
print(f"Gain : x{t_unitaire / t_session:.1f}")
```

Question 4. Complétez le tableau suivant avec vos mesures. Le gain correspond-il à l'ordre de grandeur annoncé dans le cours ?

Méthode	Temps (ms)	Ratio
Appels unitaires	???	1×
Session persistante	???	???×

2.4. Gestion du rate limit

```
def appel_avec_rate_limit(
    session : requests.Session,
    url : str,
    params : dict = None,
    delai_ms : int = 200,
) -> dict:
    """
    Appel GET avec lecture des headers de rate limit.
    Attend automatiquement si le quota est presque épuisé.
```

```

"""
r = session.get(url, params=params, timeout=10)

# Lire le quota restant (varie selon les APIs)
remaining = int(r.headers.get("X-RateLimit-Remaining", 999))

if r.status_code == 429:
    wait = int(r.headers.get("Retry-After", 60))
    print(f"Rate limit atteint -- pause de {wait}s...")
    time.sleep(wait)
    return appel_avec_rate_limit(session, url, params, delai_ms)

r.raise_for_status()

# Pause proactive si quota bas
if remaining < 10:
    time.sleep(1.0)
else:
    time.sleep(delai_ms / 1000)

return r.json()

```

Question 5. Appelez `appel_avec_rate_limit()` sur le endpoint `/departements/59/communes` avec `delai_ms=100`. La réponse est-elle identique à celle de `get_communes_nord()` ? Affichez les headers de réponse avec `dict(r.headers)` et identifiez les headers de rate limit présents (s'il y en a).

2.5. Activation du cache

```

# Activer le cache SQLite -- 24h de TTL pour les communes (donnees stables)
requests_cache.install_cache(
    "tp3_cache",
    backend = "sqlite",
    expire_after = 86_400, # 24 heures
)

# Tester : premier appel (reseau) puis deuxieme appel (cache)
import time

t0 = time.perf_counter()
r1 = session.get(f"{GEO_BASE}/departements/59/communes", timeout=10)
t1 = time.perf_counter() - t0

t0 = time.perf_counter()
r2 = session.get(f"{GEO_BASE}/departements/59/communes", timeout=10)
t2 = time.perf_counter() - t0

print(f"1er appel (reseau) : {t1*1000:.1f} ms")
print(f"2eme appel (cache) : {t2*1000:.1f} ms")
print(f"Gain cache : x{t1/t2:.0f}")
print(f"Depuis cache : {getattr(r2, 'from_cache', 'N/A')}")

```

Question 6. Quel est le gain de temps entre le premier et le deuxième appel ? Dans quels cas ce cache est-il pertinent pour un pipeline de production ? Dans quels cas serait-il problématique ?

Question 7. Rédigez en Markdown une synthèse de 5 lignes résumant les 4 mécanismes de robustesse

implémentés dans cette partie (Session, Retry, rate limit, cache) et leur rôle respectif dans un pipeline de production.

3. Validation Pydantic

3.1. Modéliser la réponse de l'API communes

```
class CommuneAPI(BaseModel):
    """Modele de validation pour un item de /departements/59/communes."""
    nom : str
    code : str = Field(min_length=5, max_length=5)
    population : Optional[int] = None # peut etre absent ou null
    codesPostaux : list[str] = []

    @field_validator("code")
    @classmethod
    def code_dept_59(cls, v: str) -> str:
        """Verifie que le code INSEE correspond bien au departement 59."""
        if not v.startswith("59"):
            raise ValueError(f"Code INSEE inattendu pour le dept 59 : {v}")
        return v

    @field_validator("population")
    @classmethod
    def population_positive(cls, v):
        if v is not None and v < 0:
            raise ValueError(f"Population negative impossible : {v}")
        return v
```

Question 8. Testez votre modèle sur les 3 cas suivants. Pour chaque cas, prédissez d'abord le comportement (succès ou `ValidationError`), puis exécutez et comparez.

```
cas_tests = [
    # Cas normal
    {"nom": "Lille", "code": "59350",
     "population": 233098, "codesPostaux": ["59000"]},
    # Population absente (null JSON)
    {"nom": "Petit-Village", "code": "59543",
     "population": None, "codesPostaux": ["59177"]},
    # Code INSEE invalide (mauvais departement)
    {"nom": "Paris", "code": "75056",
     "population": 2161000, "codesPostaux": ["75001"]},
]

for cas in cas_tests:
    try:
        commune = CommuneAPI(**cas)
        print(f"OK : {commune.nom} -- pop={commune.population}")
    except ValidationError as e:
        print(f"ERR : {cas['nom']} -- {e.errors()[0]['msg']}")
```

Question 9. Appliquez le modèle `CommuneAPI` à la liste complète `communes_raw` en mode *fail soft* : les items valides alimentent un `DataFrame` Pandas, les items invalides sont collectés dans une liste

de rejets avec le message d'erreur Pydantic. Affichez le nombre de valides, le nombre de rejetés, et exportez les rejets dans un fichier `rejets_communes.csv`.

Indication :

```
valides, rejets = [], []
for item in communes_raw:
    try:
        c = CommuneAPI(**item)
        valides.append(c.model_dump())
    except ValidationError as e:
        rejets.append({"item": str(item), "erreur": str(e)})

df_communes = pd.DataFrame(valides)
print(f"Valides : {len(valides)} | Rejetes : {len(rejets)}")
```

3.2. Détecter une rupture de schéma

Une API peut changer son format sans prévenir. Simulez ce scénario pour comprendre comment Pydantic vous protège.

```
# Simulation : l'API renomme "population" en "pop" dans une nouvelle version
communes_v2 = [
    {**item, "pop": item.pop("population", None)}
    for item in communes_raw[:10]
]

# "population" n'existe plus -- est-ce que CommuneAPI le detecte ?
for item in communes_v2[:3]:
    try:
        c = CommuneAPI(**item)
        print(f"OK : {c.nom} -- pop={c.population}")
    except ValidationError as e:
        print(f"ERR : champ manquant ou invalide -- {e.errors()[0]['msg']}")
```

Question 10. Que se passe-t-il? `population` est déclaré `Optional[int] = None` – pourquoi le modèle **ne lève pas** d'erreur même si le champ a changé de nom? En quoi est-ce un problème? Comment le rendre plus strict si on veut détecter ce changement?

Indication : Réfléchissez à la différence entre `Optional[int] = None` (la valeur peut être absente) et un champ obligatoire `int` (le champ doit être présent).

Question 11. Ajoutez un deuxième modèle `CommuneAPIStrict` dans lequel `population` est **obligatoire** (`int`, sans valeur par défaut). Testez-le sur `communes_v2`. Comparez le comportement avec `CommuneAPI`. Quand préférer chaque approche?

3.3. Modèle pour une transaction DVF

Question 12. Créez un modèle Pydantic `TransactionDVF` pour valider les lignes de votre DataFrame DVF. Le modèle doit comporter au minimum les champs suivants :

- `valeur_fonciere` : float, obligatoirement entre 1 000 € et 5 000 000 €.
- `date_mutation` : date Python (parsée depuis une string ISO).
- `type_local` : str, obligatoire.
- `surface_reelle_bati` : float optionnel, strictement positif si présent.

- `nom_commune` : str, obligatoire.
- Un validator `date_pas_future` qui rejette toute date postérieure à aujourd'hui.

Appliquez ce modèle en mode fail soft sur 500 lignes de votre DataFrame DVF et calculez le taux de lignes valides.

4. Géocodage Nominatim

4.1. Fonction de géocodage unitaire

Contraintes Nominatim – à respecter absolument

- **User-Agent obligatoire** : toute requête sans User-Agent est bloquée. Utilisez un identifiant réel.
- **1 requête par seconde maximum** : la dépasser entraîne un blocage IP. Toujours appeler `time.sleep(1)` après chaque requête.
- **Pas de géocodage en masse sans cache** : re-géocoder les mêmes adresses est inutile et antisocial envers le service public.
- **Usage non commercial seulement** : pour un usage commercial, utiliser l'API payante ou héberger Nominatim soi-même.

```
NOMINATIM_URL = "https://nominatim.openstreetmap.org/search"
NOM_HEADERS = {
    "User-Agent": "UCLille-TP3/1.0 (votre@email.ucl.fr)",
    "Accept-Language": "fr",
}

def geocoder_adresse(adresse: str) -> dict | None:
    """
    Geocode une adresse textuelle via Nominatim.
    Retourne {"lat": float, "lon": float, "display_name": str} ou None.
    IMPORTANT : appeler time.sleep(1) apres chaque invocation.
    """
    r = requests.get(
        NOMINATIM_URL,
        params = {
            "q" : adresse,
            "format" : "json",
            "limit" : 1,
            "countrycodes": "fr",
            "addressdetails": 0,
        },
        headers = NOM_HEADERS,
        timeout = 15,
    )
    r.raise_for_status()
    results = r.json()

    if not results:
        return None

    return {
```

```

        "lat" : float(results[0]["lat"]),
        "lon" : float(results[0]["lon"]),
        "display_name" : results[0]["display_name"],
    }

# Test unitaire
adresse_test = "1 Place du Gnral de Gaulle, Lille, France"
result = geocoder_adresse(adresse_test)
print(f"Adresse : {adresse_test}")
print(f"Resultat : lat={result['lat']:.4f}, lon={result['lon']:.4f}")
time.sleep(1) # TOUJOURS apres chaque appel

```

Question 13. Testez `geocoder_adresse()` sur les 4 adresses suivantes. Pour chaque résultat, affichez lat, lon et display_name. Respectez bien la pause d'1 seconde entre chaque appel.

```

adresses_test = [
    "Grand Place, Lille, France",
    "Place de la Rpublique, Roubaix, France",
    "15 rue Jean Jaurès, Valenciennes, France",
    "adresse volontairement invalide xyz 99999",
]

```

Commentez en Markdown : Nominatim a-t-il trouvé toutes les adresses ? Comment gère-t-il l'adresse invalide ? Que retourne `r.json()` dans ce cas ?

4.2. Construction des adresses DVF

Avant de géocoder, il faut construire une adresse textuelle exploitable à partir des colonnes disponibles dans le dataset DVF.

```

def construire_adresse(row) -> str:
    """
    Construit une adresse geocodable a partir d'une ligne DVF.
    Privilège le numero + nom de voie quand disponibles.
    """
    parties = []

    # Numero de voie (peut etre absent)
    if pd.notna(row.get("adresse_numero")) and str(row["adresse_numero"]).strip():
        parties.append(str(int(row["adresse_numero"])))

    # Nom de voie
    if pd.notna(row.get("adresse_nom_voie")) and str(row["adresse_nom_voie"]).strip():
        parties.append(str(row["adresse_nom_voie"]).strip().title())

    # Commune (toujours presente)
    if pd.notna(row.get("nom_commune")) and str(row["nom_commune"]).strip():
        parties.append(str(row["nom_commune"]).strip().title())

    parties.append("France")
    return " ".join(parties)

# Test
ligne_exemple = df[df["type_local"] == "Appartement"].iloc[0]
print(construire_adresse(ligne_exemple))

```

Question 14. Inspectez les colonnes disponibles dans votre DataFrame DVF pour trouver les informations d'adresse (`df.columns`). Adaptez la fonction `construire_adresse()` si nécessaire selon les colonnes réellement présentes. Appliquez-la aux 10 premières transactions de type « Appartement » et affichez les adresses construites.

4.3. Géocodage en batch avec cache fichier

Géocoder des milliers d'adresses une par une est lent (1 req/s). Un cache fichier JSON permet de ne jamais re-géocoder une adresse déjà traitée – même si le script est interrompu et relancé.

```
CACHE_PATH = Path("geocache.json")

def charger_cache() -> dict:
    """Charge le cache de géocodage depuis le fichier JSON."""
    if CACHE_PATH.exists():
        return json.loads(CACHE_PATH.read_text(encoding="utf-8"))
    return {}

def sauvegarder_cache(cache: dict) -> None:
    """Persiste le cache immédiatement après chaque nouveau géocodage."""
    CACHE_PATH.write_text(
        json.dumps(cache, ensure_ascii=False, indent=2),
        encoding="utf-8"
    )

def geocoder_batch(
    adresses : list[str],
    verbose : bool = True,
) -> list[dict | None]:
    """
    Géocode une liste d'adresses avec cache fichier et rate limiting.
    Retourne une liste de resultats (dict ou None si echec).
    """
    cache = charger_cache()
    resultats = []
    n_hit, n_miss, n_fail = 0, 0, 0

    iter_ = tqdm(adresses, desc="Géocodage") if verbose else adresses

    for adresse in iter_:
        if adresse in cache:
            resultats.append(cache[adresse])
            n_hit += 1
        else:
            result = geocoder_adresse(adresse)
            cache[adresse] = result
            sauvegarder_cache(cache) # persistance immediate
            resultats.append(result)
            if result:
                n_miss += 1
            else:
                n_fail += 1
                time.sleep(1) # rate limit Nominatim

    if verbose:
```

```
print(f"Cache HIT : {n_hit} | Nouveaux : {n_miss} | Echecs : {n_fail}")

return resultats
```

Question 15. Isolez les appartements de votre dataset. Construisez une colonne `adresse_complete` avec la fonction `construire_adresse()`. Combien d'adresses **uniques** y a-t-il? Quel est le volume total de transactions pour les 50 adresses les plus fréquentes?

```
apparts = df[df["type_local"] == "Appartement"].copy()
apparts["adresse_complete"] = apparts.apply(construire_adresse, axis=1)

print(f"Transactions : {len(apparts):,}")
print(f"Adresses uniq. : {apparts['adresse_complete'].nunique():,}")
top_adresses = apparts["adresse_complete"].value_counts().head(50)
print(top_adresses.head(10))
```

Question 16. Géocodez les **200 adresses uniques les plus fréquentes**. Utilisez `geocoder_batch()` sur la liste issue de `top_adresses.head(200).index.tolist()`. Calculez et affichez :

- le taux de succès (adresses géocodées avec succès / total);
- le nombre d'adresses en cache après la première exécution;
- le temps estimé pour géocoder toutes les adresses uniques de votre dataset complet (à 1 req/s, en déduisant les hits de cache).

Indication : relancez la cellule une deuxième fois. Le taux de cache HIT devrait être de 100% et l'exécution quasi-instantanée.

Question 17. Parmi les adresses qui ont échoué (résultat `None`), affichez les 10 premières. Quelles caractéristiques communes observez-vous? Proposez une stratégie de nettoyage ou de reformulation pour améliorer le taux de succès.

Piste : regarder si le numéro de voie est présent, si le nom de commune est correctement orthographié, si la voie existe dans OSM.

4.4. Jointure des coordonnées GPS

```
# Construire un dictionnaire adresse -> coordonnees depuis le cache
cache_geo = charger_cache()

adresses_top200 = top_adresses.head(200).index.tolist()
geo_dict = {
    adr: cache_geo[adr]
    for adr in adresses_top200
    if adr in cache_geo and cache_geo[adr] is not None
}

# Rejoindre sur le DataFrame appartements
apparts["geo"] = apparts["adresse_complete"].map(geo_dict)
apparts["lat"] = apparts["geo"].apply(lambda x: x["lat"] if x else None)
apparts["lon"] = apparts["geo"].apply(lambda x: x["lon"] if x else None)

n_geo = apparts["lat"].notna().sum()
print(f"Appartements geocodes : {n_geo:,} / {len(apparts):,}")
print(f"Couverture : {n_geo/len(apparts)*100:.1f} %")
```

Question 18. Vérifiez que les coordonnées sont géographiquement cohérentes. Calculez les bornes de la boîte englobante (min/max de lat et lon) et vérifiez qu'elles correspondent bien au département du Nord. Tracez un histogramme de la distribution de `lat` et `lon` séparément.

Indication : le département 59 est approximativement compris entre lat 50.0 et 51.1, lon 2.5 et 4.2.

5. Pipeline enrichi et carte interactive

5.1. Enrichissement démographique via l'API

Vous allez reconstruire l'enrichissement démographique du TP2, mais cette fois en consommant directement l'API `geo.api.gouv.fr` avec votre client robuste – sans fichier CSV téléchargé manuellement.

```
import unicodecode

def normalize_commune(s: str) -> str:
    """Normalise un nom de commune pour la jointure."""
    if pd.isna(s): return ""
    return unicodecode.unicodecode(str(s)).upper().strip().replace(" ", "-")

# Construire df_communes depuis la liste validee par Pydantic
df_communes["cle"] = df_communes["nom"].apply(normalize_commune)

# Construire la cle dans les appartements
apparts["cle_commune"] = apparts["nom_commune"].apply(normalize_commune)

# Deduplication de securite avant jointure
df_communes_dedup = df_communes.drop_duplicates(subset="cle")
n_doublons = len(df_communes) - len(df_communes_dedup)
print(f"Doublons sur cle commune : {n_doublons}")

# Jointure
n_avant = len(apparts)
apparts = pd.merge(
    apparts,
    df_communes_dedup[["cle", "population"]].rename(columns={"population": "pop_commune"}),
    left_on = "cle_commune",
    right_on = "cle",
    how = "left",
)
n_apres = len(apparts)
taux = apparts["pop_commune"].notna().mean() * 100

print(f"Lignes avant : {n_avant:,} | Apres : {n_apres:,}")
print(f"Taux de correspondance : {taux:.1f} %")
```

Question 19. Comparez ce taux de correspondance avec celui obtenu en TP2 (fichier INSEE). Est-il meilleur ou moins bon ? Quelles communes sans correspondance apparaissent le plus souvent ? Proposez une explication.

5.2. Assemblage du pipeline final

```

def pipeline_complet(df_source: pd.DataFrame) -> pd.DataFrame:
    """
    Pipeline enrichi complet :
    1. Isolation des appartements propres
    2. Enrichissement communes via API
    3. Jointure population
    4. Jointure coordonnees GPS (depuis cache)
    5. Nettoyage final
    """
    print("=== PIPELINE DVF ENRICHI ===")

    # 1. Appartements valides avec prix_m2
    df = df_source[df_source["type_local"] == "Appartement"].copy()
    df["prix_m2"] = df["valeur_fonciere"] / df["surface_reelle_bati"]
    df = df[df["prix_m2"].between(500, 15_000)].copy()
    print(f"[1] Appartements valides : {len(df):,}")

    # 2. Adresse complete
    df["adresse_complete"] = df.apply(construire_adresse, axis=1)

    # 3. Normalisation commune
    df["cle_commune"] = df["nom_commune"].apply(normalize_commune)

    # 4. Population (depuis df_communes deja valide)
    df = pd.merge(
        df,
        df_communes_dedup[["cle", "population"]].rename(
            columns={"population": "pop_commune"}),
        left_on="cle_commune", right_on="cle", how="left"
    )
    print(f"[3] Population : {df['pop_commune'].notna().mean()*100:.1f} % de correspondance")

    # 5. Coordonnees GPS (depuis cache)
    cache_geo = charger_cache()
    df["lat"] = df["adresse_complete"].map(
        lambda a: cache_geo.get(a, {}).get("lat") if cache_geo.get(a) else None
    )
    df["lon"] = df["adresse_complete"].map(
        lambda a: cache_geo.get(a, {}).get("lon") if cache_geo.get(a) else None
    )
    pct_geo = df["lat"].notna().mean() * 100
    print(f"[4] Geocodage : {pct_geo:.1f} % d'appartements avec coordonnees")

    # 6. Sauvegarde
    df.to_parquet("dvf_enrichi_final.parquet", index=False)
    print(f"[5] Sauvegarde : dvf_enrichi_final.parquet ({len(df):,} lignes)")
    return df

df_final = pipeline_complet(df)
df_final.describe()

```

Question 20. Exécutez le pipeline et vérifiez les résultats. Combien d'appartements ont à la fois lat, lon ET pop_commune renseignés ? Quel pourcentage cela représente-t-il ? Rédigez en Markdown

un tableau récapitulatif des taux de couverture par enrichissement.

5.3. Carte interactive avec Folium

```
# Sous-ensemble geocode pour la carte
df_carte = df_final.dropna(subset=["lat", "lon", "prix_m2"]).copy()

# Centrer la carte sur le departement 59
centre = [df_carte["lat"].median(), df_carte["lon"].median()]
carte = folium.Map(location=centre, zoom_start=11, tiles="CartoDB positron")

# Colorier les points selon le prix au m2
# Seuils quartiles
q1 = df_carte["prix_m2"].quantile(0.25)
med = df_carte["prix_m2"].quantile(0.50)
q3 = df_carte["prix_m2"].quantile(0.75)

def couleur_prix(prix):
    if prix < q1: return "#15803D" # vert = bon marche
    if prix < med: return "#1E8FA0" # teal = milieu bas
    if prix < q3: return "#D97706" # amber = milieu haut
    return "#B91C1C" # rouge = cher

for _, row in df_carte.head(500).iterrows(): # limiter pour la perf
    folium.CircleMarker(
        location = [row["lat"], row["lon"]],
        radius = 4,
        color = couleur_prix(row["prix_m2"]),
        fill = True,
        fill_opacity = 0.7,
        popup = folium.Popup(
            f"<b>{row.get('nom_commune','?')}</b><br>"
            f"{row['prix_m2']:.0f} /m<br>"
            f"Surface : {row.get('surface_reelle_bati','?')} m<br>"
            f"Prix total : {row['valeur_fonciere']:.0f} ",
            max_width=200
        ),
    ).add_to(carte)

# Legende manuelle
legende = ""
<div style='position:fixed;bottom:50px;left:50px;z-index:1000;
        background:white;padding:10px;border:2px solid grey;border-radius:5px'>
<b>Prix au m</b><br>
<span style='color:#15803D'></span> Moins de {q1:.0f} <br>
<span style='color:#1E8FA0'></span> {q1:.0f} {med:.0f} <br>
<span style='color:#D97706'></span> {med:.0f} {q3:.0f} <br>
<span style='color:#B91C1C'></span> Plus de {q3:.0f}
</div>
"".format(q1=q1, med=med, q3=q3)
carte.get_root().html.add_child(folium.Element(legende))

carte.save("dvf_carte.html")
print("Carte sauvegardee : dvf_carte.html")
carte # afficher dans le notebook
```

Question 21. Décrivez en Markdown les patterns géographiques observés sur la carte. Les quartiers chers de Lille sont-ils visibles ? Y a-t-il un gradient centre-périphérie ? Identifiez deux communes avec un prix médian nettement différent de leurs voisines et proposez une explication.

Question 22. Améliorez la carte en ajoutant une couche de **cercles agrégés par commune** : pour chaque commune avec au moins 10 transactions géocodées, tracez un cercle dont le rayon est proportionnel au volume de ventes et la couleur au prix médian. Utilisez les centroïdes de latitude et longitude pour positionner les cercles.

Indication :

```
agg_commune = (
    df_carte.groupby("nom_commune")
    .agg(n_ventes=("prix_m2", "count"),
         prix_med=("prix_m2", "median"),
         lat_med =("lat", "median"),
         lon_med =("lon", "median"))
    .query("n_ventes >= 10")
    .reset_index()
)
```

6. Analyse et bilan

6.1. Analyse géospatiale des prix

Question 23. À partir du DataFrame `df_final` enrichi, répondez aux trois questions suivantes en combinant `groupby` et visualisations.

1. **Gradient urbain :** Calculez le prix médian au m² par taille de commune (catégories : rural < 5 000 hab., petite ville 5 000–20 000, ville 20 000–100 000, grande ville > 100 000). Tracez un boxplot avec Seaborn. La relation prix-taille est-elle monotone ?
2. **Saisonnalité géographique :** Le trimestre le plus actif varie-t-il selon la taille de commune ? Calculez le nombre de transactions par trimestre et par catégorie de commune. Commentez.
3. **Surface et prix :** Calculez la corrélation entre `surface_reelle_bati` et `prix_m2` pour les appartements. Est-elle positive ou négative ? Expliquez ce résultat économiquement.

Question 24. Calculez pour chaque commune géocodée (au moins 5 transactions) :

- le prix médian au m² ;
- le volume de ventes ;
- la surface médiane.

Identifiez les 5 communes avec le prix médian le plus élevé et les 5 avec le prix le plus bas. Ces résultats sont-ils cohérents avec votre connaissance du marché immobilier du Nord ?

6.2. Bilan du pipeline

Question 25. Mesurez les performances de votre pipeline en complétant le tableau suivant. Pour chaque étape, indiquez le temps d'exécution (mesuré avec `time.perf_counter()`), le nombre de lignes en entrée et en sortie, et le taux de couverture obtenu.

Étape	Temps (s)	Lignes in	Lignes out	Couverture
Chargement DVF	???	–	???	100 %
API communes	???	–	???	???
Jointure pop.	???	???	???	???
Cache géo.	0	???	???	???
Pipeline final	???	???	???	???

Question 26. Rédigez en Markdown un bilan technique structuré de votre pipeline. Pour chaque mécanisme implémenté (Session, Retry, cache, Pydantic, géocodage), indiquez :

- le problème qu’il résout ;
- la mesure concrète qui prouve son efficacité (valeurs issues de vos benchmarks) ;
- ses limites dans ce contexte précis.

7. Rapport technique (optionnel)

Cette partie est optionnelle

Elle n’est pas comptabilisée dans la note de base. Elle est fortement conseillée si vous avez terminé les parties 1 à 5 – elle démontre votre capacité à synthétiser et à communiquer des résultats techniques à un public non spécialiste.

Question 27. Rédigez un rapport de 200 mots maximum à l’attention de votre responsable à la foncière. Organisez-le autour des trois livrables produits :

1. **Pipeline API robuste** : décrivez en termes métier ce que le pipeline fait, à quelle fréquence il pourrait tourner, et pourquoi il est fiable (sans utiliser les mots « Session » ou « Retry »).
2. **Carte interactive** : décrivez les deux ou trois enseignements principaux que votre responsable peut tirer de la carte sur le marché immobilier du Nord.
3. **Limites et recommandations** : identifiez la principale limite de votre approche (taux de géocodage, couverture des communes, etc.) et proposez une action concrète pour l’améliorer.

Le texte doit être compréhensible par une personne qui ne code pas et ne connaît pas Python.

Conseils de présentation

- Structurez le notebook avec des titres correspondant aux sections de ce sujet.
- Pour chaque question, rédigez une cellule Markdown de réponse sous le code – expliquez vos choix, pas seulement les résultats.
- Soignez les graphiques : titre, axes étiquetés, unités (€/m², nombre de transactions).
- Vérifiez que le notebook s’exécute du début à la fin sans erreur avant de rendre.
- Le fichier `geocache.json` doit être fourni avec le rendu – il permet de vérifier que le géocodage a bien été effectué.